Table 1: End-to-end execution time (s, median per iteration) and speedup split by pass type. ADT fields = non-recursive fields annotated with `@BENCH adt_fields`; SoA bufs = 1+ non-recursive field slots across all constructors (recursive children are stored in the tag buffer). Speedup >1× means SoA is faster; **bold** marks >1.1×.

Program	**ADT**	**SoA**	**Fold passes**			**Map passes**		
	fields	bufs	AoS (s)	SoA (s)	Speedup	AoS (s)	SoA (s)	Speedup
Compiler	9	8	0.1386	0.0533	**2.60**×	0.1626	0.1793	0.91×
DBQuery	15	13	0.1200	0.0889	**1.35**×	0.1352	0.1666	0.81×
DomTree	15	14	0.1371	0.0621	**2.21**×	0.0582	0.0818	0.71×
KDTree	17	16	0.1915	0.1404	**1.36**×	–	–	–
LinearListReduction	11	11	0.0296	0.0051	**5.85**×	–	–	–
List	2	2	0.3258	0.3034	1.07×	0.2533	0.3414	0.74×
MonoTree	3	2	0.0176	0.0221	0.80×	0.0608	0.0419	**1.45**×
ObjectGraph	5	4	0.0933	0.0795	**1.17**×	0.1461	0.1612	0.91×
OctTree_sumMass	16	–	0.0353	0.0272	**1.30**×	–	–	–
OctTree_sumEnergy	16	–	0.0354	0.0317	**1.12**×	–	–	–
OctTree_countActive	16	–	0.0364	0.0293	**1.25**×	–	–	–
OctTree_countParticles	16	–	0.0361	0.0273	**1.32**×	–	–	–
OctTree_barnesHutPotential	16	–	0.0415	0.0395	1.05×	–	–	–
OctTree_fmmPotential	16	–	0.0963	0.0954	1.01×	–	–	–
OctTree_scaleEnergy	16	–	–	–	–	0.1471	0.1761	0.84×
OctTree_clearFlags	16	–	–	–	–	0.1506	0.1754	0.86×
PiecewiseFunctions	8	7	0.1416	0.0915	**1.55**×	0.2407	0.2710	0.89×
TernaryTree	5	3	0.0296	0.0285	1.04×	0.0839	0.0772	1.09×
Trie	10	9	0.0638	0.0324	**1.97**×	0.1539	0.1702	0.90×
ColorOctree	25	18	0.0934	0.0767	**1.22**×	–	–	–

Table 2: Per-pass performance for `Compiler`, ADT has 9 fields, SoA uses 8 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	**T**	**Uses**	**Dead%**	**AoS med±err**	**SoA med±err**	**Speedup**
blockCountPass	F	2/9	78%	0.0122	**0.0019**	**6.32**×
branchStatsPass	F	2/9	78%	0.0119	**0.0034**	**3.47**×
castInstCountPass	F	3/9	67%	0.0184	**0.0013**	**13.67**×
has cycle	F	4/9	56%	0.0349	**0.0311**	**1.12**×
instCountPass	F	2/9	78%	0.0122	**0.0020**	**6.06**×
latencyModelPass	F	3/9	67%	0.0121	**0.0030**	**4.02**×
memoryOpStatsPass	F	3/9	67%	0.0120	**0.0038**	**3.14**×
stripSideEffectsPass	M	7/9	22%	0.0818	0.0903	0.91×
targetReturnPass	M	9/9	0%	0.0808	0.0890	0.91×
throughputModelPass	F	3/9	67%	0.0122	**0.0030**	**4.03**×
verifyIR	F	9/9	0%	0.0127	**0.0036**	**3.57**×
Total				0.3012	0.2326	**1.30**×
Geomean						**3.15**×

Table 3: Per-pass performance for `DBQuery`, ADT has 15 fields, SoA uses 13 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	**T**	**Uses**	**Dead%**	**AoS med±err**	**SoA med±err**	**Speedup**
clearQueryFlags	M	14/15	7%	**0.0676**	0.0828	0.82×
countJoins	F	3/15	80%	0.0187	**0.0116**	**1.61**×
filterSelectivitySkew	F	5/15	67%	0.0188	**0.0123**	**1.52**×
hashJoinPressure	F	5/15	67%	0.0191	**0.0129**	**1.49**×
scaleCosts	M	15/15	0%	**0.0677**	0.0838	0.81×
sumCost	F	6/15	60%	0.0188	**0.0153**	**1.23**×
sumMemory	F	6/15	60%	0.0185	**0.0147**	**1.25**×
sumRows	F	6/15	60%	0.0261	**0.0220**	**1.19**×
Total				0.2552	0.2555	1.00×
Geomean						**1.20**×

Table 4: Per-pass performance for `DomTree`, ADT has 15 fields, SoA uses 14 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	**T**	**Uses**	**Dead%**	**AoS med±err**	**SoA med±err**	**Speedup**
SumArea	F	6/15	60%	0.0352	**0.0197**	**1.79**×
computeWidths	M	14/15	7%	**0.0295**	0.0457	0.65×
count styled	F	3/15	80%	0.0340	**0.0117**	**2.90**×
find max Bottom	F	5/15	67%	0.0340	**0.0192**	**1.77**×
scaleLayout	M	15/15	0%	**0.0287**	0.0360	0.80×
sumTextWidth	F	3/15	80%	0.0339	**0.0115**	**2.94**×
Total				0.1953	0.1439	**1.36**×
Geomean						**1.55**×

Table 5: Per-pass performance for `KDTree`, ADT has 17 fields, SoA uses 16 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	**T**	**Uses**	**Dead%**	**AoS med±err**	**SoA med±err**	**Speedup**
Find nearest Neighbour	F	13/17	24%	0.0299	**0.0212**	**1.41**×
countInRange tight_box	F	13/17	24%	0.0299	**0.0202**	**1.48**×
photonMappingBenchmark	F	12/17	29%	0.0441	0.0412	1.07×
pointCloudNeighborhood	F	11/17	35%	0.0269	**0.0167**	**1.61**×
sumMassInRange	F	14/17	18%	0.0295	**0.0203**	**1.45**×
twoPointCorrelation bin_8_16	F	11/17	35%	0.0312	**0.0208**	**1.50**×
Total				0.1915	0.1404	**1.36**×
Geomean						**1.41**×

Table 6: Per-pass performance for `LinearListReduction`, ADT has 11 fields, SoA uses 11 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med±err	SoA med±err	Speedup
reduction	F	2/11	82%	0.0296	**0.0051**	**5.85**×
Total				0.0296	0.0051	**5.85**×
Geomean						**5.85**×

Table 7: Per-pass performance for `List`, ADT has 2 fields, SoA uses 2 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med±err	SoA med±err	Speedup
add1 List	M	2/2	0%	**0.2533**	0.3414	0.74×
length List	F	1/2	50%	0.0484	**0.0269**	**1.80**×
sumList	F	2/2	0%	0.1384	0.1401	0.99×
sumList tail recursive	F	2/2	0%	0.1390	0.1364	1.02×
Total				0.5791	0.6448	0.90×
Geomean						1.08×

Table 8: Per-pass performance for `MonoTree`, ADT has 3 fields, SoA uses 2 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med±err	SoA med±err	Speedup
add1Tree	M	3/3	0%	0.0608	**0.0419**	**1.45**×
sumTree	F	3/3	0%	**0.0091**	0.0113	0.81×
sumTree TailRec	F	3/3	0%	**0.0084**	0.0108	0.78×
Total				0.0784	0.0640	**1.23**×
Geomean						0.97×

Table 9: Per-pass performance for `ObjectGraph`, ADT has 5 fields, SoA uses 4 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med±err	SoA med±err	Speedup
countLargeItems	F	3/5	40%	0.0131	**0.0105**	**1.24**×
countMarked	F	3/5	40%	0.0132	**0.0106**	**1.25**×
countSurvivors	F	4/5	20%	0.0140	0.0129	1.09×
deadBytes	F	4/5	20%	0.0136	**0.0123**	**1.10**×
liveBytes	F	4/5	20%	0.0136	0.0124	1.10×
sumObjIds	F	3/5	40%	0.0125	**0.0104**	**1.21**×
sweepUnmarked	M	5/5	0%	0.0730	0.0809	0.90×
totalHeapSize	F	3/5	40%	0.0133	**0.0104**	**1.27**×
touchHotObjects	M	5/5	0%	0.0731	0.0803	0.91×
Total				0.2394	0.2407	0.99×
Geomean						**1.11**×

Table 10: Per-pass performance for `PiecewiseFunctions`, ADT has 8 fields, SoA uses 7 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med±err	SoA med±err	Speedup
addConstPW	M	8/8	0%	**0.1219**	0.1383	0.88×
autorefineMaxLevel	F	4/8	50%	0.0219	**0.0191**	**1.14**×
compressMass	F	3/8	62%	0.0198	**0.0117**	**1.70**×
diffPW	M	8/8	0%	**0.1187**	0.1327	0.89×
lbDeuxLoadProxy	F	5/8	38%	0.0218	**0.0185**	**1.18**×
norm2Estimate	F	5/8	38%	0.0255	**0.0179**	**1.42**×
pmapCutHistogram	F	4/8	50%	0.0323	**0.0127**	**2.55**×
truncateTolViolations	F	3/8	62%	0.0202	**0.0115**	**1.75**×
Total				0.3823	0.3625	1.05×
Geomean						**1.36**×

Table 11: Per-pass performance for `TernaryTree`, ADT has 5 fields, SoA uses 3 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med±err	SoA med±err	Speedup
add 1 tree	M	5/5	0%	0.0839	0.0772	1.09×
sum tree	F	5/5	0%	0.0296	0.0285	1.04×
Total				0.1135	0.1057	1.07×
Geomean						1.06×

Table 12: Per-pass performance for `Trie`, ADT has 10 fields, SoA uses 9 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med±err	SoA med±err	Speedup
autocompleteTopKProxy	F	5/10	50%	0.0149	**0.0085**	**1.76**×
countLazyNodes	F	4/10	60%	0.0126	**0.0066**	**1.92**×
countTerminals	F	3/10	70%	0.0119	**0.0058**	**2.05**×
decayTrieStats	M	10/10	0%	**0.0769**	0.0864	0.89×
resetTraversalState	M	10/10	0%	0.0770	0.0838	0.92×
sumPrefixFreq	F	3/10	70%	0.0123	**0.0059**	**2.10**×
sumSubtreeHints	F	3/10	70%	0.0121	**0.0057**	**2.10**×
Total				0.2177	0.2026	1.07×
Geomean						**1.58**×